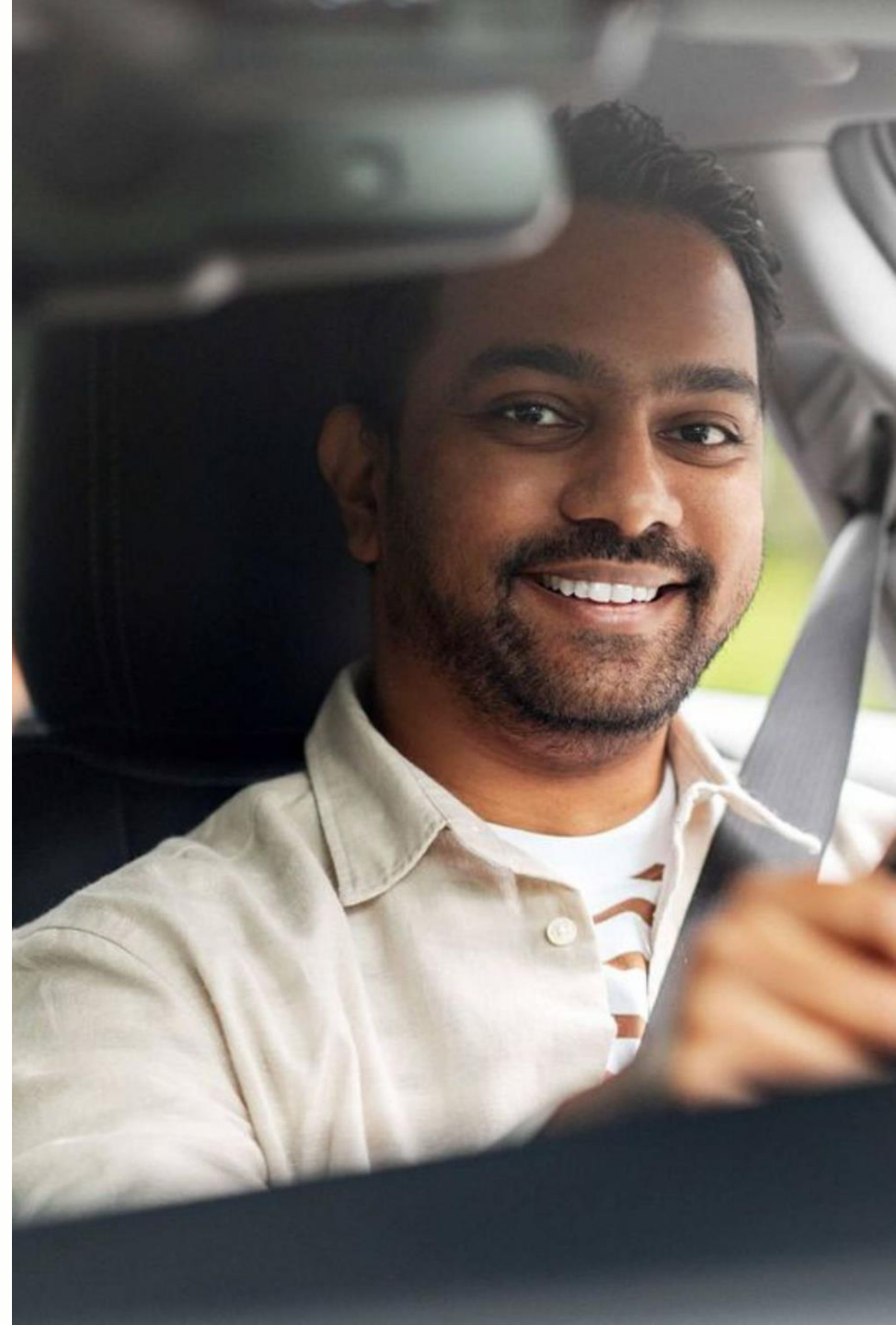


iFocus

A Machine Learning attention Monitor



What is iFocus?

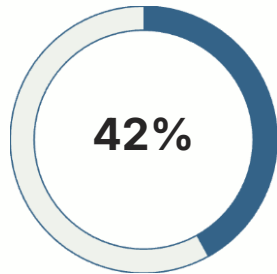
iFocus is a Machine Learning model, specially designed to monitor driver's attention through vehicle dashcams. It uses different facial landmarks to decide whether the driver is attentive or not during a trip.

Motivation.

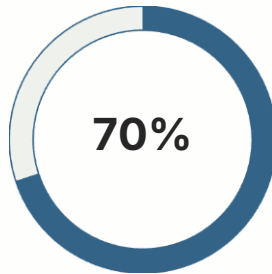
Private transportation companies are almost everywhere nowadays. They offer quick pickup, modern vehicles, and faster overall trip.

However, due to the misuse of such outstanding tool. Passenger safety has been a rising concern for most of women, children and maybe even adult men!

During the past months, passengers reported through social media that private sector transportation drivers undergo long driving hours to gain more profit and overcome their difficult financial challenges. This is a critical safety issue as the chase of more money can cost the life of both driver and passenger.



of Egypt's hail-riding commuters rely on app-based transportation services.



of full-time app drivers have around 12-16 hours of work to make actual profit

Uber

Drivers reported that they usually exceed 18 hours during work days



Careem

Some drivers have been caught on drugs to relax their bodies and endurance heavy workloads without feeling knee or back pain



inDrive

A passenger reported that his driver consumed 80% of a highway drive scrolling through the application to review incoming requests.



Didi

Drivers go overtime to complete unrealistic challenges for bonus rewards. Some passengers reported that their drivers said that they haven't slept for 2-3 days to complete these challenges.





Solution: iFocus

iFocus is a Machine Learning model built on **Mobile Net** to monitor the driver's attention through the observations of 5 face landmarks: 2 Eyes, Nose tip, and Mouth corners. We feed a trip record to the model and then classifies his behavior into 3 different classes.



Alert

The driver is awake and pays clear attention to the road



Yawning

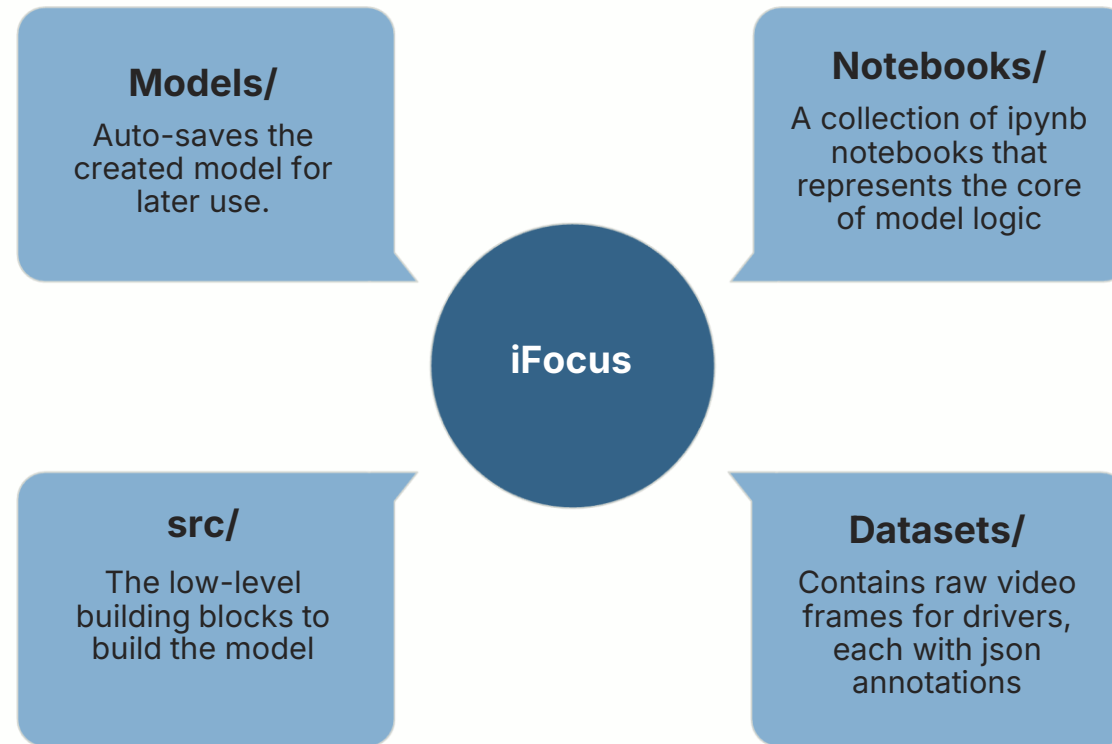
Driver is sober, but he/she is suffering from high workload



Micro-Sleeping

The driver is losing control over the car due to exhaustion or drugs

Project Structure



- **src/**

- models.py
- utils.py
- dataset.py
- config.py

- **Notebooks/**

- 01_eda_and_landmarks.ipynb
- 02_model_training.ipynb
- 03_evaluation.ipynb

src/

config.py

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent.parent
4 DATASET_DIR = BASE_DIR / "datasets" / "classification_frames"
5 MODEL_DIR = BASE_DIR / "models"
6
7 IMG_SIZE = 224
8 BATCH_SIZE = 32
9
10 LABEL_MAP = {
11     "alert": 0,
12     "microsleep": 1,
13     "yawning": 2
14 }
```

Basic model configuration

src/

config.py

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent.parent
4 DATASET_DIR = BASE_DIR / "datasets" / "classification_frames"
5 MODEL_DIR = BASE_DIR / "models"
6
7 IMG_SIZE = 224
8 BATCH_SIZE = 32
9
10 LABEL_MAP = {
11     "alert": 0,
12     "normal_sleep": 1,
13     "snoring": 2
14 }
```

Directory
names

src/

config.py

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent.parent
4 DATASET_DIR = BASE_DIR / "datasets" / "classification_frames"
5 MODEL_DIR = BASE_DIR / "models"
6
7 IMG_SIZE = 224
8 BATCH_SIZE = 32
9
10 LABEL_MAP = {
11     "alert": 0,
12     "alarmclock": 1,
13     "yawning": 2
14 }
```

Image & Batch Size

src/

config.py

```
1 from pathlib import Path
2
3 BASE_DIR = Path(__file__).resolve().parent.parent
4 DATASET_DIR = BASE_DIR / "datasets" / "classification_frames"
5 MODEL_DIR = BASE_DIR / "models"
6
7 IMG_SIZE = 224
8 BATCH_SIZE = 32
9
10 LABEL_MAP = {
11     "alert": 0,
12     "microsleep": 1,
13     "yawning": 2
14 }
```

Encoding Map

src/

dataset.py

```
1 import json
2 import pandas as pd
3 import tensorflow as tf
4 from pathlib import Path
5 from src.config import DATASET_DIR, LABEL_MAP, IMG_SIZE, BATCH_SIZE
6
7 def load_annotations(filename: str) -> pd.DataFrame:
8     with open(DATASET_DIR / filename, "r") as f:
9         annotations = json.load(f)
10
11     data = []
12     for image_path, annotation in annotations.items():
13         image_path = Path(image_path)
14         data.append({
15             "image_path": str(DATASET_DIR / image_path.relative_to("./classification_frames")),
16             "label": annotation["driver_state"],
17             "landmarks": annotation["landmarks"]
18         })
19
20     df = pd.DataFrame(data)
21     df["label_id"] = df["label"].map(LABEL_MAP)
22     return df
23
24 def _load_and_preprocess(path: str, label: int):
25     image = tf.io.read_file(path)
26     image = tf.image.decode_jpeg(image, channels=3)
27     image = tf.image.resize_with_pad(image, IMG_SIZE, IMG_SIZE)
28     return tf.cast(image, tf.float32), label
29
30 def create_tf_dataset(df: pd.DataFrame, is_training: bool = False):
31     ds = tf.data.Dataset.from_tensor_slices((df["image_path"].values, df["label_id"].values))
32     ds = ds.map(_load_and_preprocess, num_parallel_calls=tf.data.AUTOTUNE)
33
34     if is_training:
35         ds = ds.shuffle(1000)
36
37     return ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

DataFrame Creation & Manipulation

src/

dataset.py

**Converts the original
Folders + .json
dataset files to
Pandas DataFrame**

```
1 import json
2 import pandas as pd
3 import tensorflow as tf
4 from pathlib import Path
5 from src.config import DATASET_DIR, LABEL_MAP, IMAGE_SIZE, BATCH_SIZE
6
7 def load_annotations(filename: str) -> pd.DataFrame:
8     with open(DATASET_DIR / filename, "r") as f:
9         annotations = json.load(f)
10
11     data = []
12     for image_path, annotation in annotations.items():
13         image_path = Path(image_path)
14         data.append({
15             "image_path": str(DATASET_DIR / image_path.relative_to("./classification_frames")),
16             "label": annotation["driver_state"],
17             "landmarks": annotation["landmarks"]
18         })
19
20     df = pd.DataFrame(data)
21     df["label_id"] = df["label"].map(LABEL_MAP)
22     return df
23
24 def load_and_preprocess_data(n_imgs: int, label: str):
25     imgs = tf.data.Dataset.from_tensor_slices(
26         [str(DATASET_DIR / image_path) for image_path in Path(DATASET_DIR).glob(f"*{label}*")]
27     )
28     return tf.data.Dataset.from_tensor_slices(imgs).shuffle(1000).batch(BATCH_SIZE)
29
30 def create_tf_dataset(df: pd.DataFrame, is_training: bool = False):
31     ds = tf.data.Dataset.from_tensor_slices((df["image_path"].values, df["label_id"].values))
32     ds = ds.map(load_and_preprocess_data, num_parallel_calls=tf.data.experimental.DEFAULT)
33
34     if is_training:
35         ds = ds.shuffle(1000)
36
37     return ds.prefetch(1000).as_numpy_iterator()
```

src/

dataset.py

```
1 import sys
2 import os
3 import tensorflow as tf
4 from tensorflow import keras
5 from tensorflow.keras import layers
6 from tensorflow.keras import models
7
8 def load_annotations(filename):
9     with open(filename, 'r') as f:
10         annotations = json.load(f)
11
12     data = []
13     for image_path, annotation in annotations.items():
14         image_path = os.path.join(
15             'images', annotation['image_path'],
16             'image.jpg')
17         label = annotation['label']
18         data.append((image_path, label))
19
20     return data
21
22 def _load_and_preprocess(path: str, label: int):
23     image = tf.io.read_file(path)
24     image = tf.image.decode_jpeg(image, channels=3)
25     image = tf.image.resize_with_pad(image, IMG_SIZE, IMG_SIZE)
26     return tf.cast(image, tf.float32), label
27
28 def create_tf_dataset(data_loader, data_loader, data_loader):
29     ds = tf.data.Dataset.from_tensor_slices((data_loader, data_loader))
30     ds = ds.map(_load_and_preprocess, num_parallel_calls=tf.data.experimental.DEFAULT)
31     ds = ds.shuffle(1000)
32     return ds.batch(32).prefetch(tf.data.experimental.DEFAULT)
```

**Resize all images to
a uniform size
224 x 224**

src/

dataset.py

```
1 import os
2 import pandas as pd
3 import tensorflow as tf
4 from pathlib import Path
5 from tensorflow import keras
6 from tensorflow.keras import layers
7
8 def load_and_preprocess_image(path, label):
9     """Load and preprocess an image and its label.
10     Args:
11         path: Path to the image file.
12         label: Label for the image.
13     Returns:
14         A tuple containing the image as a tf.Tensor and the label as an integer.
15     """
16     image = tf.io.read_file(path)
17     image = tf.image.decode_image(image, channels=3)
18     image = tf.image.resize(image, [256, 256])
19     image = tf.cast(image, tf.float32) / 255.0
20
21     label = tf.convert_to_tensor(label)
22
23     return image, label
24
25 def load_and_preprocess_image_batch(paths, labels):
26     """Load and preprocess a batch of images and labels.
27     Args:
28         paths: List of paths to the images.
29         labels: List of labels for the images.
30     Returns:
31         A tuple containing a batch of images as a tf.Tensor and a batch of labels as a tf.Tensor.
32     """
33     images = []
34     labels = []
35     for path, label in zip(paths, labels):
36         image, label = load_and_preprocess_image(path, label)
37         images.append(image)
38         labels.append(label)
39
40     images = tf.stack(images)
41     labels = tf.stack(labels)
42
43     return images, labels
44
45 def create_tf_dataset(df: pd.DataFrame, is_training: bool = False):
46     """Create a tf.data.Dataset from a pandas DataFrame.
47     Args:
48         df: A pandas DataFrame with 'image_path' and 'label_id' columns.
49         is_training: A boolean indicating if the dataset is for training.
50     Returns:
51         A tf.data.Dataset object.
52     """
53     ds = tf.data.Dataset.from_tensor_slices((df["image_path"].values, df["label_id"].values))
54     ds = ds.map(_load_and_preprocess, num_parallel_calls=tf.data.AUTOTUNE)
55
56     if is_training:
57         ds = ds.shuffle(1000)
58
59     return ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
```

Convert image paths
into image tensors

Prefetches next batch

src/

models.py

```
1 # src/models.py
2 import tensorflow as tf
3 from tensorflow.keras import layers, models, applications
4 from src.config import IMG_SIZE
5
6 def build_mobilenet_model(num_classes: int = 3, trainable_base: bool = False):
7     inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
8
9     # 1. On-the-fly Data Augmentation
10    x = layers.RandomFlip("horizontal")(inputs)
11    x = layers.RandomRotation(0.05)(x)
12    x = layers.RandomBrightness(0.1)(x)
13
14    # 2. Backbone Preprocessing (Scales pixels to [-1, 1])
15    x = applications.mobilenet_v2.preprocess_input(x)
16
17    # 3. Backbone Feature Extractor
18    base_model = applications.MobileNetV2(
19        input_shape=(IMG_SIZE, IMG_SIZE, 3),
20        include_top=False,
21        weights="imagenet"
22    )
23    base_model.trainable = trainable_base
24
25    x = base_model(x, training=False)
26    x = layers.GlobalAveragePooling2D()(x)
27    x = layers.Dropout(0.3)(x)
28
29    # 4. Classification Output Layer
30    outputs = layers.Dense(num_classes, activation="softmax")(x)
31
32    model = models.Model(inputs, outputs, name="DriverState_MobileNetV2")
33    return model
```

Model Setup

src/

models.py

```
1 # src/models.py
2 import tensorflow as tf
3 from tensorflow.keras import layers, models, applications
4 from src.config import IMG_SIZE
5
6 def build_mobilenet_model(num_classes: int = 0, trainable_base: bool = False):
7     inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
8
9     # 1. On-the-fly Data Augmentation
10    x = layers.RandomFlip("horizontal")(inputs)
11    x = layers.RandomRotation(0.05)(x)
12    x = layers.RandomBrightness(0.1)(x)
13
14    # 2. Backbone Preprocessing (Scales pixels to [-1, 1])
15    x = applications.mobilenet_v2.preprocess_input(x)
16
17    # 3. Backbone Feature Extractor
18    base_model = applications.MobileNetV2(
19        input_shape=(IMG_SIZE, IMG_SIZE, 3),
20        include_top=False,
21        weights="imagenet"
22    )
23    base_model.trainable = trainable_base
24
25    x = base_model(x, training=False)
26    x = layers.GlobalAveragePooling2D()(x)
27    x = layers.Dropout(0.5)(x)
28
29    # 4. Classification Output Layer
30    outputs = layers.Dense(num_classes, activation="softmax")(x)
31
32    model = models.Model(inputs, outputs, name="Trained_MobileNetV2")
33    return model
```

Define Input & Data Augmentation

src/

models.py

```
1 # src/models.py
2 import tensorflow as tf
3 from tensorflow.keras import layers, models, applications
4 from src.config import IMG_SIZE
5
6 def build_mobilenet_model(num_classes: int = 1, trainable_base: bool = False):
7     inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
8
9     # 1. On-the-fly Data Augmentation
10    x = layers.RandomFlip("horizontal")(inputs)
11    x = layers.RandomRotation(0.45)(x)
12    x = layers.RandomBrightness(0.1)(x)
13
14    # 2. Backbone Preprocessing (Crops pixels to [-1, 1])
15    x = applications.mobilenet_v2.preprocess_input(x)
16
17    # 3. Backbone Feature Extractor
18    base_model = applications.MobileNetV2(
19        input_shape=(IMG_SIZE, IMG_SIZE, 3),
20        include_top=False,
21        weights="imagenet"
22    )
23    base_model.trainable = trainable_base
24
25    x = base_model(x, training=False)
26    x = layers.GlobalAveragePooling2D()(x)
27    x = layers.Dropout(0.3)(x)
28
29    # 4. Classification Output Layer
30    outputs = layers.Dense(num_classes, activation="softmax")(x)
31
32    model = models.Model(inputs, outputs, name="Distributed-MobileNetV2")
33    return model
```

MobileNet Backbone

src/

models.py

```
1 # src/models.py
2 import tensorflow as tf
3 from tensorflow.keras import layers, models, applications
4 from src.config import IMG_SIZE
5
6 def build_mobilenet_model(num_classes: int = 1, trainable_base: bool = False):
7     inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
8
9     # 1. On-the-fly Data Augmentation
10    x = layers.RandomFlip("horizontal")(inputs)
11    x = layers.RandomRotation(0.45)(x)
12    x = layers.RandomBrightness(0.1)(x)
13
14    # 2. Backbone Preprocessing (Crops pixels to [-1, 1])
15    x = applications.mobilenet_v2.preprocess_input(x)
16
17    # 3. Backbone Feature Extractor
18    base_model = applications.MobileNetV2(
19        input_shape=(IMG_SIZE, IMG_SIZE, 3),
20        include_top=False,
21        weights="imagenet"
22    )
23    base_model.trainable = trainable_base
24
25    x = base_model(x, training=False)
26    x = layers.GlobalAveragePooling2D()(x)
27    x = layers.Dropout(0.5)(x)
28
29    # 4. Classification Output Layer
30    outputs = layers.Dense(num_classes, activation="softmax")(x)
31
32    model = models.Model(inputs, outputs, name="DriverState_MobileNetV2")
33    return model
```

Classifier
Dense Layer

notebooks/

01_eda_and_landmarks.ipynb

```
1 import sys
2 sys.path.append("..")
3
4 from PIL import Image
5 from src.dataset import load_annotations
6 from src.utils import plot_landmarks
7
8
9 train_df = load_annotations("annotations_train.json")
10
11 print("Train split:\n", train_df["label"].value_counts())
12 sample = train_df.iloc[0]
13 img = Image.open(sample["image_path"])
14 print("Sample Image resolution:", img.size)
15
16 plot_landmarks(sample["image_path"], sample["landmarks"], sample["label"])
```

Exploring the dataset

notebooks/

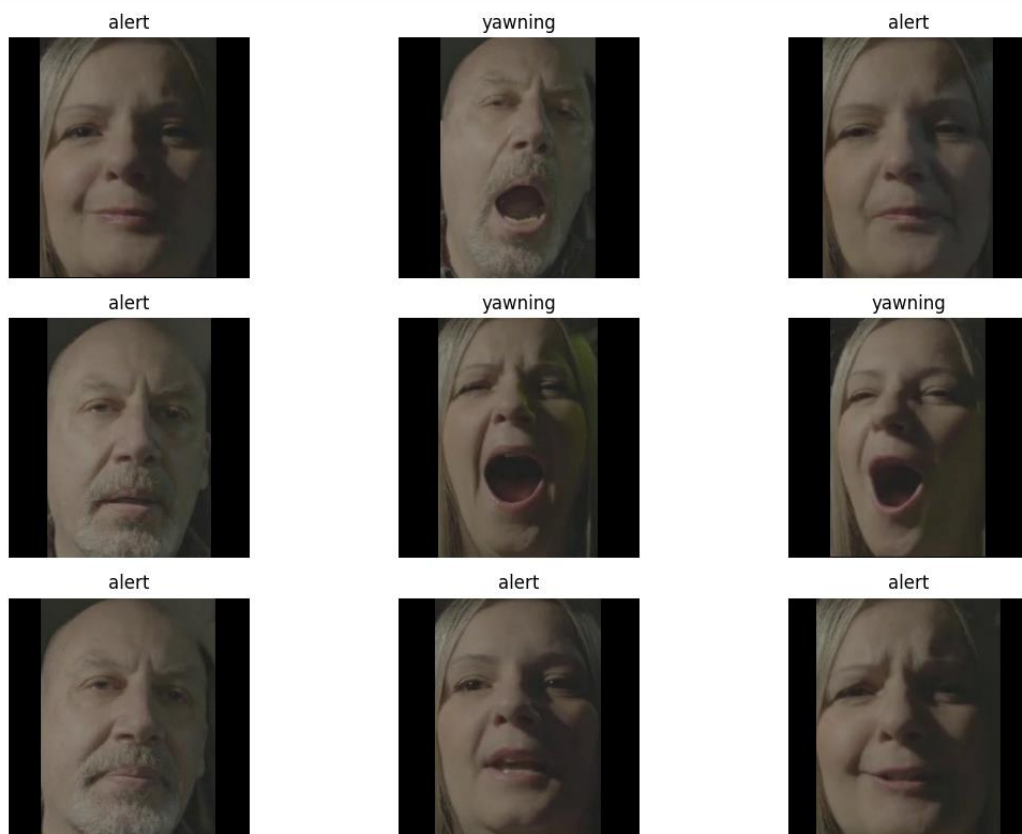
01_eda_and_landmarks.ipynb



**Exploring the
dataset**

notebooks/

01_eda_and_landmarks.ipynb



Exploring the
dataset

notebooks/

01_eda_and_landmarks.ipynb

Micro-sleep



**Exploring the
dataset**

notebooks/

02_model_training.ipynb

Actual training notebook!

```
1 import sys
2 sys.path.append("..")
3 import tensorflow as tf
4 from src.dataset import load_annotations, create_tf_dataset
5 from src.models import build_mobilenet_model
6 from src.config import MODEL_DIR
7 from sklearn.utils.class_weight import compute_class_weight
8 import numpy as np
9 train_df = load_annotations("annotations_train.json")
10 val_df = load_annotations("annotations_val.json")
11
12 train_ds = create_tf_dataset(train_df, is_training=True)
13 val_ds = create_tf_dataset(val_df, is_training=False)
14
15 weights = compute_class_weight(
16     "balanced",
17     classes=np.array([0, 1, 2]),
18     y=train_df["label_id"]
19 )
20 class_weights = dict(enumerate(weights))
21 print("Class weights:", class_weights)
22
23 model = build_mobilenet_model()
24 model.compile(
25     optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
26     loss="sparse_categorical_crossentropy",
27     metrics=["accuracy"]
28 )
29
30 MODEL_DIR.mkdir(exist_ok=True)
31 callbacks = [
32     tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
33     tf.keras.callbacks.ModelCheckpoint(MODEL_DIR / "best_mobilenet.keras", save_best_only=True)
34 ]
35
36 history = model.fit(
37     train_ds,
38     validation_data=val_ds,
39     epochs=15,
40     class_weight=class_weights,
41     callbacks=callbacks
42 )
```

notebooks/

02_model_training.ipynb

Read & Prepare The datasets

```
1 import sys
2 sys.path.append("../")
3 import tensorflow as tf
4 from src.dataset import load_annotations, create_tf_dataset
5 from src.models import build_mobilenet_model
6 from src.config import MODEL_PATH
7 from sklearn.utils.class_weight import compute_class_weight
8 import numpy as np
9 train_df = load_annotations("annotations_train.json")
10 val_df = load_annotations("annotations_val.json")
11
12 train_ds = create_tf_dataset(train_df, is_training=True)
13 val_ds = create_tf_dataset(val_df, is_training=False)
14
15 weights = compute_class_weight(
16     "balanced",
17     classes=np.array([0, 1, 2]),
18     y=train_df["label_id"]
19 )
20 class_weights = dict(enumerate(weights))
21 print("Class weights:", class_weights)
22
23 model = build_mobilenet_model()
24 model.compile(
25     optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
26     loss="sparse_categorical_crossentropy",
27     metrics=["accuracy"]
28 )
29
30 MODEL_PATH.mkdir(exist_ok=True)
31 callbacks = [
32     tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
33     tf.keras.callbacks.ModelCheckpoint(MODEL_PATH / "best_mobilenet.keras", save_best_only=True)
34 ]
35
36 history = model.fit(
37     train_ds,
38     validation_data=val_ds,
39     epochs=15,
40     class_weight=class_weights,
41     callbacks=callbacks
42 )
```

notebooks/

02_model_training.ipynb

Create & Compile the model

```
1 import sys
2 sys.path.append("../")
3 import tensorflow as tf
4 from src.dataset import load_annotations, create_tf_dataset
5 from src.models import build_mobilenet_model
6 from src.config import MODEL_PATH
7 from sklearn.utils.class_weight import compute_class_weight
8 import numpy as np
9 train_df = load_annotations("annotations_train.json")
10 val_df = load_annotations("annotations_val.json")
11
12 train_ds = create_tf_dataset(train_df, is_training=True)
13 val_ds = create_tf_dataset(val_df, is_training=False)
14
15 weights = compute_class_weight(
16     "balanced",
17     classes=np.array([0, 1, 2]),
18     y=train_df["label_id"]
19 )
20 class_weights = dict(ziperate(weights))
21 print("Class weights:", class_weights)
22
23 model = build_mobilenet_model()
24 model.compile(
25     optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
26     loss="sparse_categorical_crossentropy",
27     metrics=["accuracy"]
28 )
29
30 MODEL_PATH.mkdir(exist_ok=True)
31 callbacks = [
32     tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
33     tf.keras.callbacks.ModelCheckpoint(MODEL_PATH / "best_mobilenet.keras", save_best_only=True)
34 ]
35
36 history = model.fit(
37     train_ds,
38     validation_data=val_ds,
39     epochs=15,
40     class_weight=class_weights,
41     callbacks=callbacks
42 )
```


notebooks/

02_model_training.ipynb

Configure & Start training

```
1 import sys
2 sys.path.append("../")
3 import tensorflow as tf
4 from src.dataset import load_annotations, create_tf_dataset
5 from src.models import build_mobilenet_model
6 from src.config import MODEL_DIR
7 from sklearn.utils.class_weight import compute_class_weight
8 import numpy as np
9 train_df = load_annotations("annotations_train.json")
10 val_df = load_annotations("annotations_val.json")
11
12 train_ds = create_tf_dataset(train_df, is_training=True)
13 val_ds = create_tf_dataset(val_df, is_training=False)
14
15 weights = compute_class_weight(
16     "balanced",
17     classes=np.array([0, 1, 2]),
18     y=train_df["label_id"]
19 )
20 class_weights = dict(ziperate(weights))
21 print("Class weights:", class_weights)
22
23 model = build_mobilenet_model()
24 model.compile(
25     optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
26     loss="sparse_categorical_crossentropy",
27     metrics=["accuracy"]
28 )
29
30 MODEL_DIR.mkdir(exist_ok=True)
31 callbacks = [
32     tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True),
33     tf.keras.callbacks.ModelCheckpoint(MODEL_DIR / "best_mobilenet.keras", save_best_only=True)
34 ]
35
36 history = model.fit(
37     train_ds,
38     validation_data=val_ds,
39     epochs=15,
40     class_weight=class_weights,
41     callbacks=callbacks
42 )
```

notebooks/

03_evaluation.ipynb

```
1 import sys
2 sys.path.append("..")
3
4 import numpy as np
5 import tensorflow as tf
6 from sklearn.metrics import classification_report, confusion_matrix
7 import seaborn as sns
8 import matplotlib.pyplot as plt
9 from src.config import MODEL_DIR, LABEL_MAP
10 from src.dataset import load_annotations, create_tf_dataset
11
12 test_df = load_annotations("annotations_test.json")
13 test_ds = create_tf_dataset(test_df, is_training=False)
14
15 model = tf.keras.models.load_model(MODEL_DIR / "best_mobilenet.keras")
16 y_true = test_df["label_id"].values
17 y_pred_probs = model.predict(test_ds)
18 y_pred = np.argmax(y_pred_probs, axis=1)
19 target_names = list(LABEL_MAP.keys())
20
21 print(classification_report(y_true, y_pred, target_names=target_names))
22 cm = confusion_matrix(y_true, y_pred)
23 sns.heatmap(cm, annot=True, fmt='d', xticklabels=target_names, yticklabels=target_names, cmap='Blues')
24 plt.xlabel('Predicted')
25 plt.ylabel('Actual')
26 plt.show()
```

Testing the model

notebooks/

03_evaluation.ipynb

```
1 import sys
2 sys.path.append("../")
3
4 import numpy as np
5 import tensorflow as tf
6 from sklearn.metrics import classification_report, confusion_matrix
7 import seaborn as sns
8 import matplotlib.pyplot as plt
9 from src.config import NAMES_2018, LABELS_2018
10 from src.dataset import load_annotations, create_tf_dataset
11
12 test_df = load_annotations("annotations_test.json")
13 test_ds = create_tf_dataset(test_df, is_training=False)
14
15 model = tf.keras.models.load_model(NAMES_2018 / "test_model.keras")
16 y_true = test_ds["label_id"].values
17 y_pred_probs = model.predict(test_ds)
18 y_pred = np.argmax(y_pred_probs, axis=-1)
19 target_names = list(LABELS_2018.keys())
20
21 print(classification_report(y_true, y_pred, target_names=target_names))
22 cm = confusion_matrix(y_true, y_pred)
23 sns.heatmap(cm, annot=True, fmt="f", xticklabels=target_names, yticklabels=target_names, cmap="Blues")
24 plt.xlabel("Predicted")
25 plt.ylabel("Actual")
26 plt.show()
```

Load test dataset

notebooks/

03_evaluation.ipynb

```
1 import sys
2 sys.path.append("../")
3
4 import numpy as np
5 import tensorflow as tf
6 from sklearn.metrics import classification_report, confusion_matrix
7 import seaborn as sns
8 import matplotlib.pyplot as plt
9 from src.config import MODEL_DIR, LABEL_MAP
10 from src.dataset import load_annotations, create_tf_dataset
11
12 test_df = load_annotations("annotations_test.json")
13 test_ds = create_tf_dataset(test_df, is_training=False)
14
15 model = tf.keras.models.load_model(MODEL_DIR / "best_mobilenet.keras")
16 y_true = test_df["label_id"].values
17 y_pred_probs = model.predict(test_ds)
18 y_pred = np.argmax(y_pred_probs, axis=1)
19 target_names = list(LABEL_MAP.keys())
20
21 print(classification_report(y_true, y_pred, target_names=target_names))
22 cm = confusion_matrix(y_true, y_pred)
23 sns.heatmap(cm, annot=True, fmt='f', xticklabels=target_names, yticklabels=target_names, cmap='Blues')
24 plt.xlabel('Predicted')
25 plt.ylabel('Actual')
26 plt.show()
```

Predict using our model

notebooks/

03_evaluation.ipynb

```
1 import sys
2 sys.path.append("../")
3
4 import numpy as np
5 import tensorflow as tf
6 from sklearn.metrics import classification_report, confusion_matrix
7 import seaborn as sns
8 import matplotlib.pyplot as plt
9 from src.config import NAMES_DIR, LABELS_MAP
10 from src.dataset import load_annotations, create_tf_dataset
11
12 test_df = load_annotations("annotations_test.json")
13 test_ds = create_tf_dataset(test_df, is_training=False)
14
15 model = tf.keras.models.load_model(NAMES_DIR / "test_model.keras")
16 y_true = test_df["label_id"].values
17 y_pred_probs = model.predict(test_ds)
18 y_pred = np.argmax(y_pred_probs, axis=-1)
19 target_names = list(LABELS_MAP.keys())
20
21 print(classification_report(y_true, y_pred, target_names=target_names))
22 cm = confusion_matrix(y_true, y_pred)
23 sns.heatmap(cm, annot=True, fmt='d', xticklabels=target_names, yticklabels=target_names, cmap='Blues')
24 plt.xlabel('Predicted')
25 plt.ylabel('Actual')
26 plt.show()
```

Report metrics

notebooks/

03_evaluation.ipynb



precision	recall	f1-score	support	
alert	0.95	0.94	0.94	1728
microsleep	0.77	0.85	0.81	416
yawning	0.98	0.94	0.96	358
accuracy		0.92		2502
macro avg	0.90	0.91	0.90	2502
weighted avg	0.93	0.92	0.92	2502

demo



1/1 ————— 1s 1s/step

Prediction: micro-sleep (84.68% confidence)

PS C:\Users\vip comp\Desktop\Orange X ischool\Final Project\iFocus>

How can we benefit from iFocus?

iFocus can be integrated with transportation mobile apps as an evidence if a passenger faced a life-threatening trip.



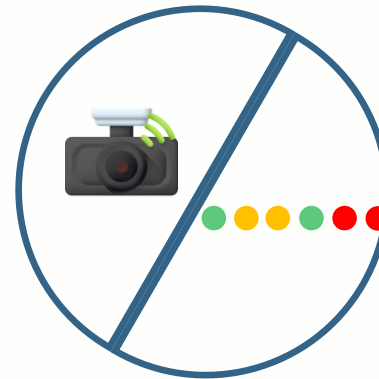
The passenger chooses his favorite application

No special platform. iFocus is a raw machine learning model and not biased by any company



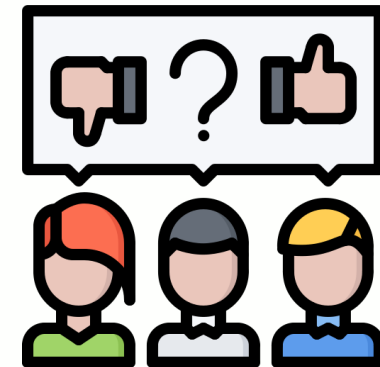
A passenger can report the irresponsible driver

This feature is almost available across all applications. The passenger can clarify his problem along with the complaint to help the model.



iFocus starts to scan the trip

This can be done using a dashcam offered by the company or an auto-start video feature from the driver's phone.



Make Decision

The company can make the decision of banning the driver, send him warnings, or mark them safe. A more advanced option is to share a database of the banned drivers to prevent the driver from registering in another application

iFocus variants: Where else can we use it?

Focus is important in many other aspects. iFocus features a modular structure that allows fine-tuning and parameters change so it can take many forms and monitor different things



Online interviews

We can tune iFocus to monitor different facial reactions such as: Looking away, using phone, following up screen.



Online Classrooms

iFocus can also be tuned to monitor student's attention during online courses. It will report attention level for every student so the lecturer can get insights that improves teaching quality



Employee Exhaustion

Tech companies can attach webcams or use the employees' laptops to monitor their exhaustion levels based on workloads.

📌 **Keynote:** These variants will have to use different datasets to capture accurate detections. Online Classrooms and Employee Exhaustion can use DAISEE dataset for example, while Online interviews can use a simpler dataset due to fewer labels.

Thank You!

Any Questions?

